Rocket attachment expansion plan

Extend the existing image workflow into general files, reuse across chats, direct capture, and image annotation. Keep every attachment separate from the prompt in the transcript and keep importing, previewing and sending distinct actions.

Status and sequence

Phase	Deliverable	Status
1	PDF, text and code attachments	Built: cards, side-panel preview, provider delivery as text; real model turns not yet run
2	Copy to chat, Save as and Reveal stored file	Built to the approved attachment reuse mockup
3	Browser, window, other-app, screen and region capture, with a pen	Built to the approved screen capture mockup
4	Crop and annotate images	Planned

The image upload baseline is implemented: paste, file selection, per-pane drop targets, preview, restart persistence, queue restoration and provider handoff references. Phase 1 was built to the approved general file attachment mockup; the README's "Images and files in chat" describes what shipped for phases 1 and 2. Phase 3 starts with its mockup.

Each phase proceeds through mockup review, implementation, controlled desktop checks and visual inspection. Real provider checks that send model turns require separate approval.

Shared foundation

- Extend `ImageAttachment` into a discriminated attachment union while continuing to read existing image records, drafts, transcript items and queues.
- Keep stable attachment IDs and host-owned copies of original files. SQLite stores metadata; disk stores originals, previews and derived files. The renderer uses the existing MessagePort.
- Separate import validation, preview generation, provider conversion and export. A readable preview does not by itself establish that a provider received the document.
- Preserve reference ownership across drafts, queues and sent messages. Cleanup must retain files still referenced by any chat or by an edited image's source relationship.
- Capture the destination chat or new-chat draft before asynchronous work starts. Changing focus, switching tabs or closing another pane must not redirect results.
- Include new agent-accessible controls in the shared shell map and renderer dispatcher.
- Handle files as content. Preview code as text, never execute it or render uploaded HTML as a privileged app page. Report unsupported, oversized, corrupt and missing files clearly.

Primary integration points: `src/shared/types.ts`, `src/host/attachments.ts`, `src/host/store.ts`, `src/host/hub.ts`, `src/host/handoff.ts`, provider adapters, `src/renderer/src/drafts.ts`, `Composer.tsx`, `Attachments.tsx`, `Transcript.tsx`, and the shared/renderer app maps.

Phase 1 General file attachments

Outcome: Attach PDF, text and code files through the existing paperclip, paste and drop workflow. Images continue working alongside them.

1. Agree on compact file cards, metadata, preview behavior, preparation and error states.
2. Add a general attachment union and backward-compatible persistence. Keep image limits separate from document limits; finalize document byte and extraction limits during implementation.
3. Validate content and filenames in the host, save independent copies and derive previews.
4. Add bounded plain-text/code previews and a PDF viewer. Treat password-protected PDFs, scanned pages and failed extraction explicitly; do not present an empty extraction as success.
5. Audit the installed provider contracts. Supply native document content when supported, extracted text within a budget, or a stored file path with a verified reading capability. If the selected route cannot read the attachment, retain the draft and explain the limitation.
6. Preserve file references through send, queue, Stop/Edit, session loss and provider switching.
7. Verify mixed image/document messages, image-only and file-only messages, restart recovery, long filenames and narrow panes.

Done when: A PDF and a code file can be attached, previewed, sent, reopened after restart, and used after switching providers. The visible prompt remains separate from the file cards.

First implementation slice after UI review: shared types and migration, import/storage, plain-text and code previews, then PDF preview and provider delivery. This order keeps the storage contract stable before adding document parsing and rendering dependencies.

Phase 2 Reuse attachments between chats

Outcome: Reuse a file without locating its original again.

1. Add an attachment menu with Copy to chat, Save as and Reveal stored file.
2. Let Copy to chat target an existing chat or a new chat. Append to the destination draft; preserve existing text and files and never send automatically.
3. Extend drag-to-chat to every supported file kind with a named destination highlight.
4. Use native save dialogs for exports; cancellation changes nothing. Reveal opens the stored location without treating file contents as executable instructions.
5. Preserve shared references when the source chat is deleted. Add a compact Files in this chat view only if attachment menus prove insufficient.

Done when: Copying survives restart and deleting the source chat does not remove the destination's attachment. Exported originals match the stored bytes.

Phase 3 Capture the screen into a chat

Outcome: Attach what is visible in Rocket or any other app, marked up, without leaving the chat. Mockup: screen capture.

1. A camera beside the paperclip offers Browser page, Rocket window, Another app's window, Entire screen and Select a region.
2. Browser page is the visible viewport for the first release; full-page capture is a later extension.

3. Other windows and screens come from Electron's `desktopCapturer.getSources` in main, shown in a picker with previews. It lists every window and screen on X11 (this machine's session); with PipeWire on Wayland it returns only the one source chosen in the system picker, so the picker must fall back to that.
4. Select a region freezes the whole screen (Rocket steps aside) so it can cover other apps. Account for device scale when converting the selection into source pixels.
5. Hide capture controls before capture and route the result through the normal image importer.
6. The review dialog shows the capture with a pen (red, yellow, blue, white), Eraser (paint only, never the capture) and Clear. The drawing is flattened into the attached image.
7. Retain the originating draft as the destination, with a useful filename and the source (URL or window title). Escape, Retake and Cancel change nothing; adding still does not send.

Done when: Browser, Rocket, other-window, screen and region captures contain the intended pixels, exclude the capture UI, carry any drawing, and reach the originating draft across split-pane layouts.

Phase 4 Annotate images before sending

Outcome: Point the agent at the relevant area of an image.

1. Open an editor from an image attachment preview.
2. Add crop, arrows, rectangles and highlights, with zoom, undo/redo and Reset, building on the phase 3 pen.
3. Keep edits as source-relative operations until export. Cropping preserves source pixels; zooming changes the view, and resizing uses ordinary resampling. Never regenerate screenshot text.
4. Preserve the original. Apply creates a derived attachment and replaces only the current draft's reference; cancel leaves the draft untouched. Editing a sent image creates a new draft copy.
5. Preview the actual exported bitmap before sending. Preserve transparency where appropriate.

Done when: The provider receives the image shown in the export preview, crop coordinates remain correct at different zoom levels and the original remains available.

Verification for every phase

- Correct destination with multiple panes, tab switches and asynchronous imports.
- Draft preservation on failed sends, cancellation and unsupported provider choices.
- Readable errors for large, corrupt, unsupported and missing files.
- Attachments remain separate from prompt text in sent and queued messages.
- Keyboard access, long filenames, narrow panes and previews with the browser open.
- Persistence across renderer reload, full restart and host recovery.
- Provider payload checks, handoffs and lost-session recovery with controlled adapters.
- Appropriate unit tests, typecheck and visible isolated desktop checks; inspect captures before completion.

Keep this status table current as phases finish. UI approval applies to the reviewed phase; it does not imply that unreviewed later screens or real model turns have been approved.